UNIVERSIDADE CATÓLICA DE BRASÍLIA

ANALISADOR SINTÁTICO PARA ASSEMBLY MIPS

Relatório técnico - Terceira entrega

Disciplina	Linguagens Formais, Autômatos e Compiladores
Professor	Marcelo Eustáquio
Semestre	2º semestre de 2026
Integrantes	Mickeias Charles De Oliveira Paiva - UC24202197 Tallys Araújo Dos Santos - UC24202740 Clara Naioko De Sousa Toratani - UC24201266 Renan Silvestre De Olivera Vale - UC24200227

Brasília - DF | 28 de outubro de 2026

1. Objetivo e escopo

Esta entrega acrescenta ao compilador o analisador sintático para o subconjunto de Assembly MIPS definido no enunciado. O módulo lê a sequência de tokens gravada pelo analisador léxico no arquivo `.lex`, verifica se a organização dos tokens obedece à gramática por meio de análise descendente recursiva, registra os erros com linha, coluna, token encontrado e elemento esperado, e gera a árvore sintática concreta em linguagem DOT quando o programa é válido.

A implementação foi feita integralmente em C, sem geradores de analisadores. Cada não terminal da gramática possui uma função de reconhecimento própria, e a escolha da alternativa é feita olhando apenas o token corrente. A etapa é estritamente sintática: a existência e a duplicidade de rótulos, os limites numéricos dos operandos e a validade dos códigos de `syscall` ficam para as etapas seguintes.

1.1 Fluxo de execução

Etapa	Entrada	Responsabilidade	Saída
1. Validação	Argumentos	Confere os dois argumentos, monta os nomes a partir do prefixo e abre todos os arquivos, substituindo resultados antigos	Arquivos abertos
2. Pré-processamento	`.asm`	Remove comentários, normaliza espaços fora de strings e preserva a numeração das linhas	`.pre`
3. Análise léxica	`.pre`	Executa o AFD da entrega anterior e informa se houve erros léxicos	`.lex`, `.ts`, `.lex.err`
4. Análise sintática	`.lex`	Executada somente sem erros léxicos; descida recursiva, recuperação e construção da árvore	`.sin.err`, `.dot`

1.2 Rastreabilidade da grade

Quesito	Atendimento no projeto
1. Compilação, modularidade e interface	`main.c`, `preprocessador.c/.h`, `lexico.c/.h` e `sintatico.c/.h`; função `int AnaliseSintatica(FILE*, FILE*, FILE*)`; compilação sem avisos
2. Integração e leitura dos tokens	`main` encadeia as três etapas; o sintático carrega o `.lex` preservando categoria, lexema, linha e coluna (seção 3)
3. Descida recursiva	32 funções, uma por não terminal, com decisão pelo próximo token (seção 5)
4. Seções, rótulos e diretivas	`programa`, `secao_*`, `linha_*`, `rotulo`, `diretiva_*` e `lista_inteiros`
5. Instruções e operandos	11 grupos de instruções com 36 operações; vírgulas, dois-pontos e parênteses verificados como terminais
6. Linhas e final do arquivo	EOL sintetizado a partir das linhas dos tokens; TK_EOF único; consumo integral até TK_EOF (seção 4)
7. Detecção e localização	Mensagem com linha, coluna, categoria e lexema encontrados e elemento esperado (seção 6)
8. Recuperação	Modo pânico até o EOL da linha corrente, preservando a seção ativa (seção 6.2)
9. Árvore e saídas	Árvore concreta em DOT, `.sin.err` em toda análise, `.dot` vazio para programas inválidos (seção 7)
10. Relatório e testes	Este documento, 3 programas válidos, 7 programas com erros sintáticos e 1 com erro léxico (seção 8)

2. Estruturas de dados

2.1 Token

O analisador sintático reaproveita a estrutura `Token` declarada em `lexico.h`, de forma que categoria, lexema e posição chegam ao sintático com o mesmo formato usado na etapa anterior.

```
typedef struct {
    char nome[50];       /* categoria: INS_ADD, REG, SMB_COM, EOL, TK_EOF... */
    char lexema[512];
    int linha;
    int coluna;
} Token;
```

2.2 Lista de tokens e nó da árvore

```
typedef struct {
    Token *itens;
    size_t total;
    size_t capacidade;
} ListaTokens;

typedef struct {
    char *rotulo;        /* "<nao_terminal>" ou categoria do terminal */
    char *lexema;        /* NULL em nao terminais, EOL e TK_EOF */
    int *filhos;         /* indices dos filhos, na ordem da producao */
    size_t total_filhos;
    size_t capacidade_filhos;
} No;
```

Estrutura	Finalidade
`ListaTokens`	Vetor dinâmico com todos os tokens do `.lex`, já com os marcadores EOL e TK_EOF. O enunciado permite carregar os tokens em memória antes do reconhecimento, o que simplifica a consulta ao token corrente.
`No`	Nó da árvore concreta. Os nós ficam em um vetor dinâmico e o índice de cada um é o identificador usado no DOT (`n0`, `n1`, ...). Os filhos são guardados como índices, na ordem em que os símbolos aparecem na produção.
Tabelas de grupos	Vetores constantes terminados em `NULL` (`OPS_TRES_REGISTRADORES`, `OPS_MEMORIA`, `DIRETIVAS_INTEIROS` etc.) que reúnem as categorias de cada alternativa da gramática e são consultados pela função `pertence`.

Todos os vetores crescem por duplicação e cada alocação é verificada. Uma falha de memória ou de gravação liga o indicador interno `falha`, e a função pública passa a retornar `-1`.

2.3 Estado do analisador

Variável	Conteúdo
`entrada`, `posicao`	Tokens carregados e índice do token corrente
`nos`, `total_nos`	Árvore em construção; `nos[0]` é a raiz `<programa>`
`saida_erros`, `total_erros`	Arquivo `.sin.err` recebido por parâmetro e contador de erros
`falha`	Indica falha operacional (memória ou E/S)

3. Integração com as etapas anteriores

As interfaces públicas do pré-processador (`PreProcessamento`) e do léxico (`AnaliseLexica`) foram mantidas. Foram feitos apenas três ajustes, todos pedidos pelo enunciado desta entrega:

Ajuste	Motivo
`AnaliseLexicaTeveErros()`	Nova consulta em `lexico.h` que retorna 1 quando a última análise léxica registrou algum erro. É o mecanismo que o `main` usa para decidir se a análise sintática deve ser executada. `AnaliseLexicaTeveFalha()` continua indicando apenas falhas operacionais.
Arquivo `.lex.err`	Os diagnósticos léxicos passaram de `.err` para `.lex.err`, como pede a tabela de arquivos da seção 9 do enunciado, deixando `.sin.err` para os erros sintáticos.
Linhas preservadas no `.pre`	O pré-processador agora grava a quebra de linha de todas as linhas, inclusive vazias e só com comentários. Assim, a linha informada no `.lex` e nos diagnósticos é a mesma do arquivo `.asm` original. Como o `.pre` continua sem indentação e com espaços normalizados, a coluna se refere ao conteúdo normalizado.

3.1 Leitura do arquivo .lex

Cada linha do `.lex` tem o formato `<CATEGORIA, lexema> linha coluna`. A função `interpretar_linha` procura o último `>` da linha e lê a posição depois dele; a categoria vai do `<` até a primeira sequência `", "`, e o lexema é o que sobra entre esses dois pontos. A leitura a partir do fim é necessária porque o lexema de uma string pode conter vírgulas, espaços e o próprio caractere `>`. Linhas fora desse formato são tratadas como falha operacional.

3.2 Função main

O programa recebe exatamente dois argumentos, `main.exe <arquivo_entrada.asm> <prefixo_saida>`. A partir do prefixo são montados os nomes `.pre`, `.lex`, `.sin.err` e `.dot`; o léxico gera `.ts` e `.lex.err` com o mesmo prefixo. Todos os arquivos de saída são abertos no início em modo de escrita, então resultados de execuções anteriores são sempre substituídos. Os arquivos `.pre` e `.lex` são abertos em `w+` e reposicionados no início com `fseek` antes de servirem de entrada para a etapa seguinte.

Situação	Mensagem no terminal	Código
Programa válido	`PROGRAMA SINTATICAMENTE VÁLIDO`	0
Erros sintáticos	`PROGRAMA SINTATICAMENTE INVÁLIDO`	1
Erros léxicos	`ANÁLISE SINTÁTICA NÃO EXECUTADA: EXISTEM ERROS LÉXICOS.` (o motivo também é gravado no `.sin.err` e o `.dot` fica vazio)	1
Argumentos incorretos, falha de abertura, leitura, gravação ou memória	Mensagem de erro em `stderr`	2

4. Tratamento de EOL e TK_EOF

O léxico não gera marcadores de fim de linha, e o enunciado pede que o formato do `.lex` não seja alterado. Por isso a função `carregar_tokens` sintetiza os marcadores ao montar a lista em memória:

- depois de cada token, se o próximo token estiver em outra linha (ou se não houver próximo), é inserido um `EOL` com a linha desse token e coluna igual à posição logo após o último lexema;
- como só existem tokens em linhas com conteúdo, linhas vazias e linhas apenas com comentários não produzem marcadores;
- a última linha recebe EOL mesmo quando o arquivo não termina com quebra de linha;
- a leitura para no primeiro `TK_EOF` encontrado, que é copiado uma única vez para o final da lista; se o `.lex` não tiver esse token, ele é acrescentado na linha seguinte à do último token.

Durante a análise, `avancar()` nunca passa de `TK_EOF`. Isso garante que o reconhecimento sempre termina e que a produção `<programa>` só é aceita depois de consumir toda a entrada.

As repetições `{ <linha_dados> }` e `{ <linha_codigo> }` terminam somente quando o próximo token é `DIR_DATA`, `DIR_TEXT` ou `TK_EOF` (função `fim_de_secao`). Qualquer outro token inicia uma nova linha; se ele não for permitido na seção atual, a função da linha registra o erro, em vez de tratar o token como encerramento da seção.

Exemplo

Para o programa do enunciado, os tokens `DIR_TEXT` (linha 2), `ID SMB_COL` (linha 3) e `INS_LI REG SMB_COM NUM_INT` (linha 4) resultam na sequência:

```
DIR_TEXT EOL  ID SMB_COL EOL  INS_LI REG SMB_COM NUM_INT EOL ... INS_SYSCALL EOL TK_EOF
```

5. Funções de reconhecimento

Cada não terminal corresponde a uma função `static int nome(int pai)`. A função cria o nó do não terminal como último filho de `pai`, reconhece os símbolos da produção na ordem em que aparecem e retorna 1 em caso de sucesso ou 0 se encontrar um erro. Os terminais são consumidos por `terminal(no, categoria)`, que compara a categoria do token corrente, cria a folha e avança. As sequências de uma produção são escritas como uma cadeia de `&&`, por exemplo:

```
/* <instrucao_li> ::= "li" REG "," NUM_INT */
static int instrucao_li(int pai) {
    int no = nao_terminal(pai, "<instrucao_li>");
    return terminal(no, "INS_LI") && terminal(no, "REG") &&
           terminal(no, "SMB_COM") && terminal(no, "NUM_INT");
}
```

Com a avaliação em curto-circuito, o primeiro terminal que falhar registra o erro e interrompe a produção. Repetições `{ }` são laços `while`, e partes opcionais `[ ]` são decididas com um `if` sobre o token corrente.

5.1 Correspondência entre não terminais e funções

Não terminal	Função	Decisão / observação
`<programa>`	`programa`	Exige ao menos uma seção; laço enquanto houver `.data`/`.text`; termina em TK_EOF
`<secao>`	`secao`	`DIR_DATA` ou `DIR_TEXT`
`<secao_dados>`	`secao_dados`	Laço de linhas até `fim_de_secao`
`<secao_codigo>`	`secao_codigo`	Laço de linhas até `fim_de_secao`
`<linha_dados>`	`linha_dados`	`ID`, diretiva de dados ou `.globl`; recupera em caso de erro
`<linha_codigo>`	`linha_codigo`	`ID`, `INS_*` ou `.globl`; recupera em caso de erro
`<rotulo>`	`rotulo`	`ID SMB_COL`
`<diretiva_global>`	`diretiva_global`	`DIR_GLOBL ID`
`<diretiva_dados>`	`diretiva_dados`	Inteiros, string, `.space` ou `.align`
`<diretiva_inteiros>`	`diretiva_inteiros`	`.word`, `.half`, `.byte`
`<lista_inteiros>`	`lista_inteiros`	`NUM_INT` e laço enquanto houver `SMB_COM`
`<diretiva_string>`	`diretiva_string`	`.ascii`, `.asciiz`
`<instrucao>`	`instrucao`	Escolhe o grupo pela categoria do mnemônico
`<instrucao_tres_registradores>`	`instrucao_tres_registradores`	`op REG , REG , REG`
`<op_tres_registradores>`	`op_tres_registradores`	add, addu, sub, subu, mul, and, or, xor, nor, slt
`<instrucao_dois_registradores>`	`instrucao_dois_registradores`	`op REG , REG`
`<op_dois_registradores>`	`op_dois_registradores`	move, div
`<instrucao_um_registrador>`	`instrucao_um_registrador`	`op REG`
`<op_um_registrador>`	`op_um_registrador`	mflo, mfhi, jr

<instrucao_imediato>	instrucao_imediato	addi REG , REG , NUM_INT
<instrucao_deslocamento>	instrucao_deslocamento	op REG , REG , NUM_INT
<op_deslocamento>	op_deslocamento	sll, srl
<instrucao_li>	instrucao_li	li REG , NUM_INT
<instrucao_la>	instrucao_la	la REG , ID
<instrucao_memoria>	instrucao_memoria	op REG , <endereco>
<op_memoria>	op_memoria	lw, sw, lb, sb, lh, sh
<endereco>	endereco	ID ou NUM_INT (REG)
<instrucao_desvio>	instrucao_desvio	op REG , REG , ID
<op_desvio>	op_desvio	beq, bne, blt, ble, bgt, bge
<instrucao_salto>	instrucao_salto	op ID
<op_salto>	op_salto	j, jal
<instrucao_sem_operandos>	instrucao_sem_operandos	syscall, nop

5.2 Funções auxiliares

Função	Descrição
carregar_tokens	Lê o .lex, insere EOL e garante um único TK_EOF
interpretar_linha	Separa categoria, lexema, linha e coluna de uma linha do .lex
atual, verificar, pertence, avancar	Consulta e consumo do token corrente
terminal, terminal_do_grupo	Consomem um terminal esperado (ou um terminal de um grupo) criando a folha
erro_sintatico	Formata e grava o diagnóstico no .sin.err
recuperar	Modo pânico até o fim da linha corrente
novo_no, nao_terminal, adicionar_filho	Construção da árvore
escrever_dot, escrever_escapado	Geração do arquivo DOT com escape de aspas e barras
AnaliseSintatica	Interface pública: reinicia o estado, carrega os tokens, reconhece, grava as saídas, libera a memória e devolve 0, 1 ou -1

6. Diagnóstico e recuperação de erros

6.1 Formato das mensagens

Quando o token corrente não é o esperado, `erro_sintatico` grava uma linha no formato:

```
Linha L, coluna C: Encontrado CATEGORIA("lexema"); esperado ELEMENTO.
```

A linha e a coluna são as do token encontrado. Se o elemento ausente estava no final da linha, o token encontrado é o EOL sintetizado, e a mensagem informa o término da linha com a coluna logo após o último lexema, por exemplo `Encontrado EOL (término da linha 2)`. No final do arquivo a mensagem informa `Encontrado TK_EOF`. O elemento esperado é exibido com a categoria e o texto do terminal (`SMB_COM(",")`, `DIR_TEXT(".text")`) ou com a categoria/construção (`REG`, `NUM_INT`, `ID ou NUM_INT (endereço)`).

Para facilitar a correção, alguns casos recebem uma observação: uma instrução dentro de `.data` ou uma diretiva de dados dentro de `.text` gera a indicação de que a construção só é permitida na outra seção. Dois inteiros sem vírgula em uma lista geram `esperado SMB_COM(",") ou EOL`.

O exemplo do enunciado, `add $t0 $t1, $t2`, produz `Encontrado REG("$t1"); esperado SMB_COM(",")`, como pode ser visto no caso de teste `invalidos/05_pontuacao`.

6.2 Recuperação

A recuperação fica nas funções de linha (`linha_dados`, `linha_codigo`), nos cabeçalhos de seção e no tratamento de linhas anteriores à primeira seção. Quando a produção falha, `recuperar()` descarta os tokens restantes da linha:

```
static void recuperar(void) {
    while (!verificar(TOKEN_EOF) && !inicio_de_secao()) {
        int era_eol = verificar(TOKEN_EOL);
        avancar();
        if (era_eol) {
            return;
        }
    }
}
```

- se o token corrente já for EOL, apenas esse marcador é consumido;
- se for TK_EOF, a recuperação termina sem consumir nada;
- se aparecer `.data` ou `.text`, a recuperação para antes dele, e o laço da seção reconhece a mudança e prossegue;
- como o laço para no primeiro EOL, nenhum token de uma linha posterior é descartado.

A seção ativa não muda durante a recuperação, porque o laço de linhas da seção continua de onde parou. Toda chamada consome ao menos o token que provocou o erro ou chega a um token que encerra o laço (diretiva de seção ou TK_EOF), então não há repetição indefinida do mesmo diagnóstico. Como a recuperação vai até o fim da linha, é registrado no máximo um erro por linha, o que o enunciado permite; erros em linhas seguintes continuam sendo detectados. Qualquer erro registrado torna o programa inválido.

Arquivos vazios ou só com comentários chegam ao sintático contendo apenas TK_EOF e são rejeitados pela produção `<programa>`. Linhas anteriores à primeira seção geram um erro por linha, esperando `.data` ou `.text`.

7. Árvore sintática concreta

A árvore é montada durante o próprio reconhecimento. Cada função cria o nó do seu não terminal antes de reconhecer os filhos, e cada terminal consumido vira uma folha ligada ao nó da produção em andamento. Com isso:

- a raiz é `<programa>` (`n0`) e os identificadores seguem a ordem de criação dos nós (pré-ordem);
- os filhos ficam na ordem dos símbolos da produção, e as repetições aparecem como filhos sucessivos (as várias `<linha_codigo>` de uma seção, os pares `, NUM_INT` de uma lista);
- partes opcionais ausentes não geram nós, já que a função correspondente não é chamada;
- as folhas mostram a categoria e o lexema (`REG\n$t0`); EOL e TK_EOF aparecem apenas com a categoria;
- aspas e barras invertidas dos lexemas são escapadas, o que é necessário para strings como `"Resultado: \"ok\"\n"`.

O arquivo usa `graph [ordering=out]` para manter a ordem dos filhos na renderização. O DOT só é gravado quando a análise termina sem erros; para programas inválidos o arquivo permanece vazio, e nenhuma árvore parcial é produzida.

Para o programa de exemplo do enunciado (caso `validos/01_exemplo_text_direto`), o DOT gerado tem os mesmos 43 nós (`n0` a `n42`) e as mesmas arestas do exemplo da especificação. O arquivo completo está no Apêndice A.

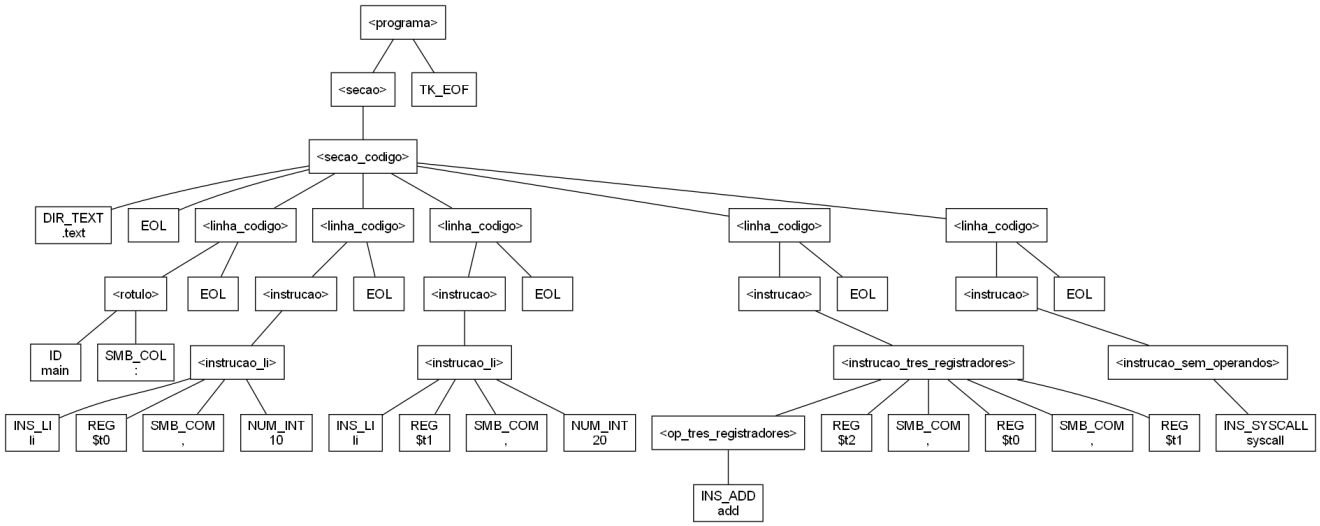


Figura 1 - Árvore sintática concreta do caso validos/01_exemplo_text_direto, renderizada com `dot -Tpng`.

As árvores dos casos 02 (221 nós) e 03 (327 nós) foram renderizadas da mesma forma e acompanham a entrega em `testes/saidas_geradas`, junto com os respectivos arquivos DOT.

8. Casos de teste

Os testes ficam em `testes/validos` e `testes/invalidos`. O script `testes/executar_testes.ps1` compila o projeto, executa todos os casos com o prefixo `testes/saidas_geradas/<tipo>_<nome>` e renderiza as árvores dos programas válidos. Os casos usados para avaliar erros sintáticos são todos lexicamente válidos; o caso 08 tem erros léxicos de propósito, para demonstrar que a análise sintática não é executada.

8.1 Resultados esperados e obtidos

Caso	Cobertura principal	Tokens	Esperado	Obtido	Erros	Nós
validos/01_exemplo_text_direto	Programa iniciado por `.text`; exemplo do enunciado	19	Válido	Válido (0)	0	43
validos/02_dados_e_diretivas	Seções `.data` e `.text`; todas as diretivas; endereços por ID e por deslocamento	104	Válido	Válido (0)	0	221
validos/03_instrucoes_alternancia	Todas as instruções; seção vazia; alternância; sem quebra final; rótulo inexistente e duplicado	155	Válido	Válido (0)	0	327
invalidos/01_arquivo_vazio	Arquivo vazio	1	Inválido	Inválido (1)	1	-
invalidos/02_sem_secao	Programa sem abertura de seção	13	Inválido	Inválido (1)	2	-
invalidos/03_secao_incorreta	Instrução em `.data`; diretiva de dados em `.text`	26	Inválido	Inválido (1)	4	-
invalidos/04_operandos	Operandos ausentes, excedentes, fora de ordem e de categoria errada	54	Inválido	Inválido (1)	12	-
invalidos/05_pontuacao	Vírgula, dois-pontos e parênteses ausentes; lista terminada em vírgula; tokens adicionais	51	Inválido	Inválido (1)	12	-
invalidos/06_recuperacao	Erros em linhas distintas e em seções alternadas, com construções válidas entre eles	56	Inválido	Inválido (1)	5	-
invalidos/07_somente_comentarios	Arquivo composto só por comentários	1	Inválido	Inválido (1)	1	-
invalidos/08_erro_lexico	Erros léxicos impedem a análise sintática	12	Não executa	Não executa (1)	-	-

A coluna *Tokens* conta as linhas do `.lex`, incluindo TK_EOF, e o número entre parênteses é o código de saída do programa. Resultado: 11 de 11 casos com o comportamento esperado. Os programas válidos geraram `Nenhum erro sintático encontrado.` e um DOT não vazio; os inválidos geraram diagnósticos e DOT vazio.

8.2 Cobertura exigida

Categoria	Onde é coberta
Organização	`.text` direto (v01); `.data` e `.text` (v02); seção `.data` vazia e alternância `.data`/`.text` (v03)

Rótulos	Rótulo isolado (`main:` em v01, `isolado:` em v02); rótulo com instrução (`inicio: add` em v03); rótulo com diretiva (`numero: .word` em v02)
Diretivas	As dez diretivas da gramática em v02, inclusive `.globl` nas duas seções
Instruções	As 36 operações dos 11 grupos, distribuídas entre v02 (li, la, memória) e v03 (demais grupos)
Operandos	Decimais, negativos e hexadecimais; strings com escapes; listas de inteiros; referências a identificadores
Memória	`lw $t0, numero` e `lw $t1, 4($sp)`, `sh $t3, 0x10($gp)` (v02)
Linhas	Linhas vazias, comentários de linha inteira e no fim da linha; v03 não termina com quebra de linha
Entrada inválida	i01 (vazio), i02 (sem seção), i07 (só comentários)
Seção incorreta	i03 e última linha de dados em i06
Estrutura incorreta	i04
Pontuação	i05 e `.word 2,, 3` em i06
Consumo completo	`add $t0, $t1, $t2, $t3` e `div $t0, $t1, $t2` (i04); `syscall $t0` e `nop nop` (i05)
Recuperação	Todos os inválidos têm erros em várias linhas; i06 mostra construções válidas reconhecidas entre os erros, inclusive após troca de seção
Limites da etapa	v03 contém `j rotulo_inexistente` e o rótulo `fim:` declarado duas vezes, e é aceito

8.3 Diagnósticos gerados

Conteúdo integral dos arquivos `.sin.err`. Os programas correspondentes estão no Apêndice B; as linhas citadas são as do arquivo `.asm`.

invalidos/01_arquivo_vazio

```
Linha 1, coluna 1: Encontrado TK_EOF; esperado DIR_DATA(".data") ou DIR_TEXT(".text").

Total de erros sintáticos: 1
```

invalidos/02_sem_secao

```
Linha 2, coluna 1: Encontrado INS_LI("li"); esperado DIR_DATA(".data") ou DIR_TEXT(".text").
Linha 3, coluna 1: Encontrado INS_ADD("add"); esperado DIR_DATA(".data") ou DIR_TEXT(".text").

Total de erros sintáticos: 2
```

invalidos/03_secao_incorreta

```
Linha 3, coluna 1: Encontrado INS_LI("li"); esperado ID, diretiva de dados ou DIR_GLOBL(".globl") (inst
ruções só são permitidas na seção .text).
Linha 4, coluna 6: Encontrado INS_ADD("add"); esperado diretiva de dados ou EOL (instruções só são perm
itidas na seção .text).
Linha 6, coluna 1: Encontrado DIR_WORD(".word"); esperado ID, instrução ou DIR_GLOBL(".globl") (diretiv
as de dados só são permitidas na seção .data).
Linha 7, coluna 6: Encontrado DIR_ASCIIZ(".asciiz"); esperado instrução ou EOL (diretivas de dados só s
ão permitidas na seção .data).

Total de erros sintáticos: 4
```

invalidos/04_operandos

```
Linha 2, coluna 13: Encontrado EOL (término da linha 2); esperado SMB_COM(",").
Linha 3, coluna 9: Encontrado ID("mensagem"); esperado NUM_INT.
Linha 4, coluna 18: Encontrado SMB_COM(","); esperado EOL.
Linha 5, coluna 9: Encontrado EOL (término da linha 5); esperado SMB_COM(",").
Linha 6, coluna 11: Encontrado NUM_INT("5"); esperado REG.
Linha 7, coluna 15: Encontrado REG("$t2"); esperado NUM_INT.
Linha 8, coluna 9: Encontrado NUM_INT("10"); esperado ID.
Linha 9, coluna 10: Encontrado ID("fim"); esperado REG.
Linha 10, coluna 3: Encontrado REG("$ra"); esperado ID.
Linha 11, coluna 4: Encontrado ID("fim"); esperado REG.
Linha 12, coluna 5: Encontrado EOL (término da linha 12); esperado REG.
Linha 13, coluna 13: Encontrado SMB_COM(","); esperado EOL.

Total de erros sintáticos: 12
```

invalidos/05_pontuacao

```
Linha 2, coluna 17: Encontrado EOL (término da linha 2); esperado NUM_INT.
Linha 3, coluna 16: Encontrado NUM_INT("2"); esperado SMB_COM(",") ou EOL.
Linha 4, coluna 5: Encontrado DIR_WORD(".word"); esperado SMB_COL(":").
Linha 5, coluna 13: Encontrado EOL (término da linha 5); esperado STRING.
Linha 7, coluna 9: Encontrado REG("$t1"); esperado SMB_COM(",").
Linha 8, coluna 11: Encontrado REG("$sp"); esperado SMB_OPA("(").
Linha 9, coluna 14: Encontrado EOL (término da linha 9); esperado SMB_CPA(")").
Linha 10, coluna 9: Encontrado SMB_OPA("("); esperado ID ou NUM_INT (endereço).
Linha 11, coluna 7: Encontrado EOL (término da linha 11); esperado SMB_COL(":").
Linha 12, coluna 9: Encontrado REG("$t0"); esperado EOL.
Linha 13, coluna 5: Encontrado INS_NOP("nop"); esperado EOL.
Linha 14, coluna 7: Encontrado EOL (término da linha 14); esperado ID.

Total de erros sintáticos: 12
```

invalidos/06_recuperacao

```
Linha 3, coluna 12: Encontrado SMB_COM(","); esperado NUM_INT.
Linha 7, coluna 13: Encontrado EOL (término da linha 7); esperado SMB_COM(",").
Linha 9, coluna 10: Encontrado SMB_OPA("("); esperado EOL.
Linha 11, coluna 9: Encontrado REG("$t1"); esperado SMB_COM(",").
Linha 14, coluna 1: Encontrado INS_NOP("nop"); esperado ID, diretiva de dados ou DIR_GLOBL(".globl") (i
nstruções só são permitidas na seção .text).

Total de erros sintáticos: 5
```

Em i06, as linhas 2, 4, 6, 8, 10, 13 e 16 são válidas e foram reconhecidas normalmente entre os erros, inclusive depois das trocas de seção nas linhas 12 e 15.

invalidos/07_somente_comentarios

```
Linha 4, coluna 1: Encontrado TK_EOF; esperado DIR_DATA(".data") ou DIR_TEXT(".text").

Total de erros sintáticos: 1
```

invalidos/08_erro_lexico

Arquivo `.lex.err`:

```
<ERRO_IDENTIFICADOR_MALFORMADO, 12abc> 2 9
<ERRO_REGISTRADOR_INVALIDO, $t9x> 3 5
```

Arquivo `.sin.err`:

```
Análise sintática não executada: existem erros léxicos (consulte testes\saidas_geradas\invalidos_08_err
o_lexico.lex.err).
```

Programas válidos

```
Nenhum erro sintático encontrado.
```

9. Compilação e execução

```
gcc main.c preprocessador.c lexico.c sintatico.c -o main.exe

main.exe <arquivo_entrada.asm> <prefixo_saida>
main.exe teste.asm resultado

dot -Tpng resultado.dot -o resultado.png
```

A execução do exemplo gera `resultado.pre`, `resultado.lex`, `resultado.ts`, `resultado.lex.err`, `resultado.sin.err` e `resultado.dot`. O projeto também compila sem avisos com `gcc -std=c11 -Wall -Wextra -Wpedantic -Wconversion -Wshadow` (GCC 6.3.0, MinGW).

Para executar todos os casos de teste no Windows:

```
powershell -ExecutionPolicy Bypass -File testes\executar_testes.ps1
```

10. Conclusão

O analisador sintático reconhece todo o subconjunto definido pela gramática com uma função por não terminal, decide as alternativas olhando apenas o próximo token e verifica a quantidade, a categoria e a ordem dos operandos, além de vírgulas, dois-pontos e parênteses. Os marcadores EOL e TK_EOF são produzidos internamente sem alterar o formato do `.lex`, e a entrada é sempre consumida até TK_EOF. Os erros são informados com linha, coluna, token encontrado e elemento esperado, e a recuperação no fim da linha permite encontrar erros nas construções seguintes sem descartar tokens de outras linhas. Para programas válidos é gerada a árvore sintática concreta completa em DOT, e o exemplo do enunciado é reproduzido nó a nó. Os 11 casos de teste tiveram o resultado esperado.

Apêndice A - DOT do caso validos/01_exemplo_text_direto

Programa:

```
# Programa iniciado diretamente por .text (exemplo do enunciado)
.text
main:
    li $t0, 10
    li $t1, 20
    add $t2, $t0, $t1
    syscall
```

Arquivo `validos_01_exemplo_text_direto.dot` gerado:

```
digraph ArvoreSintatica {
    graph [ordering=out];
    node [shape=box, fontname="Arial"];
    edge [arrowhead=none];

    n0 [label="<programa>"];
    n1 [label="<secao>"];
    n2 [label="<secao_codigo>"];
    n3 [label="DIR_TEXT\n.text"];
    n4 [label="EOL"];
    n5 [label="<linha_codigo>"];
    n6 [label="<rotulo>"];
    n7 [label="ID\nmain"];
    n8 [label="SMB_COL\n:"];
    n9 [label="EOL"];
    n10 [label="<linha_codigo>"];
    n11 [label="<instrucao>"];
    n12 [label="<instrucao_li>"];
    n13 [label="INS_LI\nli"];
    n14 [label="REG\n$t0"];
    n15 [label="SMB_COM\n,"];
    n16 [label="NUM_INT\n10"];
    n17 [label="EOL"];
    n18 [label="<linha_codigo>"];
    n19 [label="<instrucao>"];
    n20 [label="<instrucao_li>"];
    n21 [label="INS_LI\nli"];
    n22 [label="REG\n$t1"];
    n23 [label="SMB_COM\n,"];
    n24 [label="NUM_INT\n20"];
    n25 [label="EOL"];
    n26 [label="<linha_codigo>"];
    n27 [label="<instrucao>"];
    n28 [label="<instrucao_tres_registradores>"];
    n29 [label="<op_tres_registradores>"];
    n30 [label="INS_ADD\nadd"];
    n31 [label="REG\n$t2"];
    n32 [label="SMB_COM\n,"];
    n33 [label="REG\n$t0"];
    n34 [label="SMB_COM\n,"];
    n35 [label="REG\n$t1"];
    n36 [label="EOL"];
    n37 [label="<linha_codigo>"];
    n38 [label="<instrucao>"];
    n39 [label="<instrucao_sem_operandos>"];
    n40 [label="INS_SYSCALL\nsyscall"];
    n41 [label="EOL"];
```

```
    n42 [label="TK_EOF"];

    n0 -> n1;
    n0 -> n42;
    n1 -> n2;
    n2 -> n3;
    n2 -> n4;
    n2 -> n5;
    n2 -> n10;
    n2 -> n18;
    n2 -> n26;
    n2 -> n37;
    n5 -> n6;
    n5 -> n9;
    n6 -> n7;
    n6 -> n8;
    n10 -> n11;
    n10 -> n17;
    n11 -> n12;
    n12 -> n13;
    n12 -> n14;
    n12 -> n15;
    n12 -> n16;
    n18 -> n19;
    n18 -> n25;
    n19 -> n20;
    n20 -> n21;
    n20 -> n22;
    n20 -> n23;
    n20 -> n24;
    n26 -> n27;
    n26 -> n36;
    n27 -> n28;
    n28 -> n29;
    n28 -> n31;
    n28 -> n32;
    n28 -> n33;
    n28 -> n34;
    n28 -> n35;
    n29 -> n30;
    n37 -> n38;
    n37 -> n41;
    n38 -> n39;
    n39 -> n40;
}
```

Apêndice B - Programas de teste

Listagem com numeração de linhas. Os DOT dos casos válidos 02 e 03 e todas as demais saídas estão em `testes/saidas_geradas`.

validos/02_dados_e_diretivas.asm

```
 1  # Secoes .data e .text, todas as diretivas, rotulos isolados e com diretiva
 2  .data
 3  .globl main
 4  numero:   .word 10
 5  vetor:    .word 10, 20, -30, 0xFF
 6  pequeno:  .byte 65
 7  valor:    .half 1000, 0x7FFF
 8  mensagem: .asciiz "Resultado: \"ok\"\n"
 9  texto:    .ascii "ABC"
10
11  buffer:   .space 100
12            .align 2
13  isolado:
14            .word 1, 2, 3          # lista de inteiros
15
16  .text
17  .globl main
18  main:     la $a0, mensagem
19            lw $t0, numero
20            lw $t1, 4($sp)
21            sw $t1, -8($fp)
22            lb $t2, pequeno
23            sb $t2, 0($a0)
24            lh $t3, valor
25            sh $t3, 0x10($gp)
26            li $v0, 10
27            syscall
```

validos/03_instrucoes_alternancia.asm

```
# Todas as instrucoes; alternancia de secoes; secao vazia; sem quebra final
.data
.text
inicio: add $t0, $t1, $t2
    addu $t0, $t1, $t2
    sub $t0, $t1, $t2
    subu $t0, $t1, $t2
    mul $t0, $t1, $t2
    and $t0, $t1, $t2
    or $t0, $t1, $t2
    xor $t0, $t1, $t2
    nor $t0, $t1, $t2
    slt $t0, $t1, $t2

    move $s0, $t0
    div $t0, $t1
    mflo $s1
    mfhi $s2
    addi $t0, $t0, -1
    sll $t1, $t1, 2
    srl $t1, $t1, 0x1F
.data
contador: .word 0
.text
laco:
    beq $t0, $zero, fim
    bne $t0, $1, laco
    blt $t0, $t1, laco
    ble $t0, $t1, laco
    bgt $t0, $t1, laco
    bge $t0, $t1, laco
    jal rotina
    j fim
rotina: jr $ra
    j rotulo_inexistente        # referencia a rotulo inexistente (semantica)
fim:
fim:                            # rotulo duplicado (semantica)
    nop
    syscall
```

invalidos/01_arquivo_vazio.asm

```
(arquivo vazio)
```

invalidos/02_sem_secao.asm

```
# Apenas comentarios e codigo antes de qualquer secao
li $t0, 1
add $t0, $t0, $t0
.text
    nop
```

invalidos/03_secao_incorreta.asm

```
1  .data
2  valor: .word 5
3      li $t0, 10
4  rot: add $t0, $t1, $t2
5  .text
6      .word 7
7  lbl: .asciiz "x"
8      nop
```

invalidos/04_operandos.asm

```
1  .text
2      add $t0, $t1
3      li $t0, mensagem
4      add $t0, $t1, $t2, $t3
5      move $t0
6      addi $t0, 5, $t1
7      sll $t0, $t1, $t2
8      la $t0, 10
9      beq $t0, fim, $t1
10     j $ra
11     jr fim
12     mflo
13     div $t0, $t1, $t2
14     nop
```

invalidos/05_pontuacao.asm

```
1  .data
2  vetor: .word 10,
3  lista: .word 1 2
4  rot .word 3
5  msg: .asciiz
6  .text
7      add $t0 $t1, $t2
8      lw $t0, 4 $sp
9      lw $t0, 4($sp
10     sw $t0, ($sp)
11 inicio
12     syscall $t0
13     nop nop
14     .globl
15     li $v0, 10
```

invalidos/06_recuperacao.asm

```
.data
a: .word 1
b: .word 2,, 3
c: .half 4
.text
main: li $t0, 1
    add $t0, $t0
    sub $t1, $t1, $t1
    lw $t0, x($sp)
    j main
    beq $t0 $t1, main
.data
d: .byte 1
    nop
.text
    syscall
```

invalidos/07_somente_comentarios.asm

```
# arquivo composto exclusivamente por comentarios

   # outro comentario
```

invalidos/08_erro_lexico.asm

```
.text
    li $t0, 12abc
    add $t9x, $t0, $t1
```